

Name: Ambika

UID : 23BCC70026

Class : 23BCC-1(A)

Experiment – 6.1

Aim

To implement a class hierarchy in Java using the concept of **inheritance**, where the base class Person has two derived classes — Student and Teacher.

Theory

In **Object-Oriented Programming (OOP)**, **inheritance** allows one class (child) to acquire properties and behaviors (methods and variables) of another class (parent).

This promotes **code reusability**, **hierarchical classification**, and **extensibility**.

- **Base Class (Parent):** Common attributes and methods are defined here.
- **Derived Class (Child):** Inherits the parent class and adds its own specific properties or methods.

In this example:

- Person is the base class containing common attributes like name and age.
- Student and Teacher are subclasses that extend Person and add additional fields like grade or subject.

Procedure

1. Define a class Person with attributes like name and age.
2. Create a constructor in Person to initialize these attributes.
3. Define methods like displayInfo() in the Person class.
4. Create subclasses:
 - a. Student → adds attribute grade and overrides displayInfo().
 - b. Teacher → adds attribute subject and overrides displayInfo().
5. Create a Main class to create objects of Student and Teacher and display their details.
6. Compile and run the program.

Code

```
// Base Class
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Subclass 1
class Student extends Person {
    String grade;

    Student(String name, int age, String grade) {
        super(name, age); // calling parent constructor
        this.grade = grade;
    }
}
```

```

@Override
void displayInfo() {
    super.displayInfo();
    System.out.println("Grade: " + grade);
}
}

// Subclass 2
class Teacher extends Person {
    String subject;

    Teacher(String name, int age, String subject) {
        super(name, age);
        this.subject = subject;
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Subject: " + subject);
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Nikhil", 20, "A");
        Teacher t1 = new Teacher("Mr. Sharma", 45, "Computer Science");

        System.out.println("Student Details:");
        s1.displayInfo();

        System.out.println("\nTeacher Details:");
        t1.displayInfo();
    }
}

```

Sample Output

Student Details:

Name: Nikhil

Age: 20

Grade: A

Teacher Details:

Name: Mr. Sharma

Age: 45

Subject: Computer Science

Learning Outcomes

By the end of this experiment, students will be able to:

1. Understand the concept of **inheritance** in OOP.
2. Differentiate between **base** and **derived** classes.
3. Use the **super** keyword to call parent class constructors or methods.
4. Implement **method overriding** to redefine inherited behavior.
5. Develop reusable and modular code using **hierarchical relationships**.